

Logic, Search and Planning: An End-to-End Explainer

The Warehouse Robot — CS F407 Artificial Intelligence

Study notes, not for submission

How to read this

This is the whole laboratory explained from first principles: what a planning problem is, what it means for an action to be *applicable*, how logic and search fit together, what the code actually does, and — the part that matters — why one of the bugs in this laboratory was invisible to *every* check that could be written in Python.

Every number quoted here was measured from the programs in `deliverables/`. Nothing is illustrative.

If you read only one section, read §7. It is the part of this laboratory that teaches something a textbook will not.

Contents

1	The one-sentence version	1
2	States: the world as a set of true things	1
2.1	The representation	1
2.2	Why <code>frozenset</code> and not <code>set</code>	2
2.3	A modelling choice worth understanding	2
3	Actions: preconditions and effects	2
3.1	Applicability: the logical step	3
3.2	Effects: the transition	3
3.3	The warehouse actions, in full	3
4	The goal: entailed, not equalled	4
4.1	Test on pop, not on generation	4
5	Search: walking a graph nobody built	4
5.1	The implicit graph	4
5.2	BFS, and what the visited set is really for	4
5.3	What changes if you swap the search, and what does not	5
6	Putting it together: the plan by hand	5
6.1	The trap in the handout	5
6.2	The plan that works	6
7	The bug that nothing inside Python could find	6
7.1	Defect 0 — the goal test written as <code>state == G</code>	6
7.2	Defect 1 — the missing <code>visited</code> set	6
7.3	Defect 2 — Move over every pair of locations	6
7.4	The pattern	7
8	Why a generated explanation proves nothing	7

9 Prolog: an independent judge	8
9.1 What Prolog is, in three lines	8
9.2 How the answers are computed (SLD resolution)	8
9.3 Why <code>can_move(a,c)</code> fails, and why that is useful	8
9.4 Adjacent is not reachable	9
9.5 The loop closed	9
9.6 The general architecture	10
10 Answers you should be able to give	10
11 The five words to remember	10

1 The one-sentence version

The whole laboratory

$$\boxed{\text{Logic} + \text{Search} = \text{Planning}}$$

Logic decides what is **possible**: given the state of the world, which actions can be performed, and what does the world look like afterwards? *Search* decides what to **try**: of all the sequences of possible actions, which one do I explore next?

A planner is a graph search in which the graph is not given to you — it is *computed*, one node at a time, by a logical test.

That last sentence is the difference from the previous module. In the search laboratory you were handed a grid: the neighbours of a cell were simply the adjacent cells. Here nobody hands you the graph. You describe the world with propositions and actions, and the edges of the graph come into existence whenever an action's preconditions happen to be satisfied.

This has a consequence that runs through everything below: **you can now get the graph itself wrong**, and no amount of searching it correctly will tell you so.

2 States: the world as a set of true things

2.1 The representation

A **proposition** is an atomic statement that is either true or false: $\text{At}(\text{Robot}, A)$, $\text{Holding}(\text{Package})$. A **state** is the set of propositions that are *true*:

$$S_0 = \{ \text{At}(\text{Robot}, A), \text{At}(\text{Package}, A) \}.$$

In code, a state is a Python **frozenset** of strings.

The closed-world assumption — the most important line in the file

Anything *not* in the set is **false**. Not “unknown”—false.

S_0 does not contain $\text{Holding}(\text{Package})$, and that is precisely how the program knows the robot's hands are empty. It does not contain $\text{At}(\text{Robot}, B)$, and that is how it knows the robot is not at B.

This is why a state of 2 propositions can pin down a world of 7 facts. It is also why, later, Prolog answering **false** to `can_move(a,c)` means “I cannot derive it”, which the system chooses to treat as “it is false”. Same assumption, same trade.

2.2 Why frozenset and not set

Because states are used as *keys*: they go into a **visited** set and a **parent** dictionary. Python sets are mutable, therefore unhashable, therefore cannot be dictionary keys. The first LLM-generated version used plain sets and died with

```
TypeError: unhashable type: 'set'
```

the moment duplicate detection was added. Immutability is not a nicety here: a state that can be mutated after being recorded as visited would silently corrupt the search.

2.3 A modelling choice worth understanding

Where does the package go when the robot picks it up? The representation says: *nowhere*. $\text{At}(\text{Package}, A)$ is deleted and $\text{Holding}(\text{Package})$ is added. While it is held, the package has **no location proposition at all**.

That sounds like a gap. It is the opposite — it is what makes **Move** correct for free:

- If the package is on the floor and the robot walks away, the package's **At** proposition is untouched, so it stays behind. Correct.
- If the package is held, there is no **At** proposition to update, so it travels with the robot. Also correct.

One **Move** definition, both behaviours, no special cases. The alternative — $\text{At}(\text{Package}, \text{Robot})$ — would need **Move** to know about the package, and would admit states where the package is both at A and in the robot's hand.

Invariant

$\text{Holding}(\text{Package})$ and $\text{At}(\text{Package}, x)$ are never both true and never both false. The package is always in exactly one of: a place, or the robot's hand.

3 Actions: preconditions and effects

An action has four sets and a name:

Pre^+	must be present in the state
Pre^-	must be absent from the state
Add	made true by the action
Del	made false by the action

3.1 Applicability: the logical step

The entailment check

$$S \models \text{Pre}(a) \iff \text{Pre}^+(a) \subseteq S \text{ and } \text{Pre}^-(a) \cap S = \emptyset.$$

In Python, one line:

```
return self.pos_pre <= state and not (self.neg_pre & state)
```

This is what “logical reasoning” means in this laboratory. It is not a theorem prover. It is a small *model-checking* step — is this formula true in this model? — performed once per action per state. Solving the warehouse performs it **70 times**.

The reason it is so cheap is that the actions are *ground*: all their variables have already been filled in, so $\text{Move}(A, B)$ and $\text{Move}(B, C)$ are separate objects. Real planners keep the schemas and unify at search time, which is more expressive and much more expensive.

3.2 Effects: the transition

$$S' = \text{Apply}(S, a) = (S \setminus \text{Del}(a)) \cup \text{Add}(a).$$

Delete first, *then* add — and why the order is specified

If a proposition appears in both Add and Del, the order decides its fate:

- delete-then-add: it **survives**;
- add-then-delete: it is **lost**.

No action in this domain has such an overlap, so both orders give identical results *here*. That is exactly why it must be fixed in the specification rather than discovered later: an underspecified detail that happens not to matter today is a bug waiting for the domain to grow. The prompt in `prompts.md` states the order explicitly so the generated code could not choose for itself, and Test G pins it down.

3.3 The warehouse actions, in full

Action	Pre ⁺	Pre [−]	Add	Del
Move(x, y) <i>connected x, y only</i>	At(R, x)	—	At(R, y)	At(R, x)
PickUp(P, x)	At(R, x), At(P, x)	Holding(P)	Holding(P)	At(P, x)
Drop(P, x)	At(R, x), Holding(P)	—	At(P, x)	Holding(P)

Note the italicised line. **The connectivity of the warehouse is not inside any action — it decides which actions exist at all.** With

$$A \text{ — } B \text{ — } C, \quad A \not\text{— } C,$$

there are four **Move** actions, not six. Remember this sentence; §7 is about what happens when it is forgotten.

PickUp's negative precondition stops the robot picking up a package it is already holding. It is the only Pre[−] in the domain, and it exists partly so that there *is* one to test.

4 The goal: entailed, not equalled

$G \subseteq S$, never $S = G$

$$G = \{ \text{At}(\text{Package}, C) \}, \quad S_4 = \{ \text{At}(\text{Package}, C), \text{At}(\text{Robot}, C) \}.$$

$S_4 \supseteq G$, so $S_4 \models G$: **goal achieved**.

$S_4 \neq G$: equality would say **failure**.

The goal is a *partial description of the world*. It constrains the package and says nothing at all about the robot. Demanding equality would demand that the final state contain the goal proposition *and nothing else* — which is unachievable, because the robot is always somewhere.

This is the single most predictable place for generated code to go wrong, which is why it was written down in Task 0 *before* any prompting, and stated explicitly in the prompt. It still had to be fixed once (Defect 0), and because it had been specified, the fix was one line.

4.1 Test on pop, not on generation

The planner checks the goal when a state comes *off* the frontier, not when it is put on. For BFS with unit-cost actions both are correct, but testing on pop means **states expanded** counts the same thing for every search strategy, so BFS and DFS numbers can be compared honestly. (In the previous module this mattered more: for A*, goal-test-on-pop is what makes the algorithm *optimal*.)

5 Search: walking a graph nobody built

5.1 The implicit graph

The action definitions define a graph without ever constructing it:

- **nodes** = states;
- **edge** $S \xrightarrow{a} S'$ exists exactly when $S \models \text{Pre}(a)$ and $S' = \text{Apply}(S, a)$.

For the warehouse, exhaustively enumerated in `results.txt` §1, the reachable graph has **12 nodes**. In general it has up to 2^n for n propositions — which is why the graph is generated lazily, on demand: the applicability test *is* the successor function.

5.2 BFS, and what the visited set is really for

```
frontier = deque([start]);  visited = {start};  parent = {start: (None,
    None)}
while frontier:
    state = frontier.popleft()                # FIFO -> shortest plans
    first
    if problem.satisfies_goal(state): return reconstruct(parent, state)
    for action in problem.actions:
        if not action.applicable(state): continue        # <- the logic
        successor = action.apply(state)                  # <- the logic
        if successor in visited: continue                # <- termination
        visited.add(successor); parent[successor] = (state, action)
        frontier.append(successor)
```

BFS returns a **shortest** plan because it explores states in order of increasing plan length and every action costs 1 — the same argument as in the search module.

The visited set is not an optimisation

It is a **termination requirement**, and the difference is not academic.

Move(A, B) and Move(B, A) form a 2-cycle. Without duplicate detection, the frontier refills faster than it drains. On a *solvable* problem this does not matter — BFS reaches the goal before it notices. On an *unsolvable* problem it never runs out of states, so it never reports failure.

	with visited	without
solvable problem	4-action plan, 8 expansions	4-action plan, 25 expansions
unsolvable problem	“no plan”, 3 expansions	still running at 50,000

Read the first row again: on the problem the handout calls Test A, the broken version and the correct version give *the same right answer*. The bug is only visible on Test B. This is the entire reason the handout insists you test the impossible case.

5.3 What changes if you swap the search, and what does not

From `results.txt` §3, with the logic untouched:

Problem	Algo	Plan	Expanded	Generated	Peak frontier
warehouse	BFS	4	8	16	3
warehouse	DFS	4	7	13	2
corridor, 6 locations	BFS	7	27	58	8
corridor, 6 locations	DFS	7	13	28	5
corridor, 8 locations	BFS	9	47	101	11
corridor, 8 locations	DFS	9	17	38	7

Both return **valid** plans, because both use the same applicability test. Only BFS returns a **shortest** one. DFS matches it here only because a corridor offers almost no wrong turns to take — give it a branching map and it will wander. DFS touches fewer states, which is the usual trade: it commits early and is rewarded when the commitment happens to be right.

The division of labour, made visible

Changing the search changed *which* plan was found and how much work it took. It did not and could not change *which plans are legal*. Logic fixes the set of legal plans; search picks one out of it.

6 Putting it together: the plan by hand

6.1 The trap in the handout

The handout suggests you “might need to reason about”

Move(A, B), Pickup($Package, B$), Move(B, C), Drop($Package, C$).

That is not a valid plan, and finding out why is the most useful five minutes in the laboratory.

Step	Action	State before	Applicable?
1	Move(A, B)	{At(R, A), At(P, A)}	yes
2	PickUp($Package, B$)	{At(R, B), At(P, A)}	no — At(P, B) is false

The robot walked to B *without the package*, because Move does not move the package. The sequence names all the right actions in a sensible order and is wrong at the second step.

The lesson, before an LLM is anywhere near the problem

“Looks reasonable” is a judgement about a *list of action names*. “Valid” is a property of the list *together with the states it passes through*. The only way to tell them apart is to carry the state along and check each precondition where the action is actually used.

6.2 The plan that works

State	Facts	Why the action was applicable
S_0	At(R, A), At(P, A)	(given)
S_1	At(R, A), Holding(P)	PickUp(P, A): At(R, A)✓ At(P, A)✓ Holding absent✓
S_2	At(R, B), Holding(P)	Move(A, B): At(R, A)✓
S_3	At(R, C), Holding(P)	Move(B, C): At(R, B)✓
S_4	At(R, C), At(P, C)	Drop(P, C): At(R, C)✓ Holding(P)✓

$S_4 \models G$. **Four actions, and four is optimal:** one PickUp and one Drop are unavoidable (only Drop adds $At(P, C)$, and only PickUp adds the Holding it requires), and two Moves are unavoidable because A and C are not adjacent. $2 + 2 = 4$.

Hold on to the number 4. It is what catches the bug in the next section.

7 The bug that nothing inside Python could find

If you remember one thing

Three defects reached working code. They are worth ranking not by how bad they look, but by **where the evidence that would catch them lives**.

7.1 Defect 0 — the goal test written as `state == G`

Symptom: “No plan found” on the obviously solvable problem. **Caught by:** Test A, instantly.
Evidence lives: inside the code.

Loud, immediate, harmless. Predicted in writing before any prompt was issued.

7.2 Defect 1 — the missing visited set

Symptom: the program hangs — but only on the *unsolvable* problem. **Caught by:** Test B.
Evidence lives: inside the code, but only if you run the right input.

Quieter. Test A passes. It is worth noting that the LLM diagnosed the cause correctly once told the symptom, and then described the fix as “an efficiency improvement” — a plausible sentence about a termination bug.

7.3 Defect 2 — Move over every pair of locations

The generated code instantiated `Move` for all six ordered pairs of locations instead of the four connected ones. So `Move(A, C)` existed.

	correct domain	defective domain
ground actions	10	12
plan	PickUp(P,A) Move(A,B) Move(B,C) Drop(P,C)	PickUp(P,A) Move(A,C) Drop(P,C)
length	4	3
passes <code>validate_plan?</code>	yes	yes
shortest, per BFS?	yes	yes
LLM asked to justify it?	—	confirmed it, correctly
possible in the warehouse?	yes	no

Why every internal check passed — and had to

Preconditions, effects, the goal test, the independent re-execution and the optimality proof are *all evaluated relative to the action set*. The action set was the thing that was wrong.

Re-executing the plan asks: *is this plan consistent with the model?* Yes, perfectly. Nobody was asking: *is the model consistent with the warehouse?*

There is no test you can write in Python, against this model, that rejects this plan. The search is exhaustively, provably correct over a state space that was described wrongly. Its correctness is not in question — its *subject* is.

How it was actually found: by reading the output and noticing **three** actions where the hand-built plan needed **four**. Not by a test. By a *number* that disagreed with something worked out beforehand — which is the entire practical value of doing Task 1 by hand before touching the computer.

7.4 The pattern

	Defect 0	Defect 1	Defect 2
kind of error	logical	algorithmic	modelling
symptom	wrong answer	hang	none
visible from inside?	yes	yes	no
found by	Test A	Test B	a human, comparing with Task 1
LLM's explanation	silent	“an optimisation”	confidently confirmed it

The finding

The LLM was reliable on everything whose correctness is judged **from inside the code**, and unreliable on the one thing whose correctness is judged **against the world** — and it was *most* confident precisely there.

The asymmetry is the finding. Not “LLMs are unreliable”.

8 Why a generated explanation proves nothing

Task 5 asks: which do you trust, the LLM's explanation, or the independently executed state transitions? The expected answer is the transitions, and it is right. This run produced a sharper version.

When the LLM was asked to justify the three-action plan, it produced a step-by-step precondition check. **Every line of it was true.** $\text{PickUp}(P, A)$ needs $\text{At}(R, A)$ and $\text{At}(P, A)$: both hold. $\text{Move}(A, C)$ needs $\text{At}(R, A)$: it holds. $\text{Drop}(P, C)$ needs $\text{At}(R, C)$ and $\text{Holding}(P)$: both hold. The final state contains $\text{At}(P, C)$.

The independent Python re-execution agreed. BFS additionally certified the plan as *shortest*. **Three confirmations, every one of them correct, every one of them useless.**

An ordering of trust

1. **An independent check against an independent source of knowledge.** Prolog's `valid_route(a, [move(a,c)])` → `false`. The only thing that caught Defect 2.
2. **Independently executed state transitions.** Catches every error *within* the model — mis-applied preconditions, mis-sequenced effects, a plan whose steps do not join up. Cannot catch an error *in* the model.
3. **A generated explanation.** Evidence of nothing. It is text conditioned on the plan, produced by the process that produced the plan, and it will be just as fluent when the plan is wrong.

The handout's own phrasing is the thing to memorise, with one clause added:

A generated explanation is not the same as an independent verification
— and an independent *execution* is only as good as the model it executes.

9 Prolog: an independent judge

9.1 What Prolog is, in three lines

A Prolog program is **facts**, **rules** and **queries**.

```
connected(a,b).                % a fact
can_move(X,Y) :- connected(X,Y). % a rule:  Connected(X,Y) -> CanMove
    (X,Y)
?- can_move(a,b).              % a query: does this follow?
true.
```

`head :- body` is the implication $body \rightarrow head$, written backwards. A query asks whether the goal is *derivable* from the facts and rules.

9.2 How the answers are computed (SLD resolution)

Prolog reasons **backwards**, from the goal:

$$\text{can_move(a,b)} \xrightarrow{\text{rule, } X \mapsto a, Y \mapsto b} \text{connected(a,b)} \xrightarrow{\text{fact}} \square \quad (\text{empty goal: success})$$

Each step replaces the current goal with the body of a matching clause, after **unification** fills in the variables. Reach the empty goal and the query succeeds. Run out of clauses and it backtracks.

For Task 8's chain, forward reasoning proves the conclusion:

$$\underbrace{\text{WetRoad}}_{\text{fact}} \implies \text{Slippery} \implies \text{ReduceSpeed}$$

by two applications of *modus ponens*. Prolog reduces the *goal* `reduce_speed` to `slippery` to `wet_road`, and finds a fact. **Same theorem, opposite direction of travel.**

9.3 Why `can_move(a,c)` fails, and why that is useful

There is no fact `connected(a,c)`, and `can_move/2` has one clause whose body is a single `connected/2` goal — it is **not recursive, therefore not transitive**. No derivation exists, so Prolog answers **false**.

“False” means “not derivable”

This is **negation as failure**, the closed-world assumption again. Classical logic would say *unknown*; Prolog says *false*.

Two consequences worth separating:

- It is what makes Prolog usable as a verifier: an action the knowledge base cannot justify is treated as *forbidden*. A verifier that answered “unknown” would verify nothing.
- It is also why Prolog is not classical logic. Its *completion* semantics reads the clauses for `can_move/2` as an if-and-only-if: $\text{CanMove}(X,Y) \leftrightarrow \text{Connected}(X,Y)$. The single implication alone would leave $\text{CanMove}(a,c)$ undetermined.

9.4 Adjacent is not reachable

```
?- can_move(a,c).      false.      % a and c are not adjacent ...
?- reachable(a,c).     true.       % ... but c IS reachable from a, via b
```

`reachable/2` is the transitive closure and needs *recursion*. These are different questions — and **Defect 2** is exactly the error of treating the second as though it were the first.

9.5 The loop closed

`run_prolog.py` translates the Python plan into a Prolog goal and asks a knowledge base that has never seen the planner:

```
the corrected planner
  Python plan          : Pickup(Package,A) -> Move(A,B) -> Move(B,C)
    -> Drop(Package,C)
  Python self-validation : VALID
  translated Prolog goal : valid_route(a, [move(a,b),move(b,c)]).
  Prolog answer         : true.                >>> AGREEMENT.

the defective planner (Move over every pair of locations)
  Python plan          : Pickup(Package,A) -> Move(A,C) -> Drop(
    Package,C)
  Python self-validation : VALID
  translated Prolog goal : valid_route(a, [move(a,c)]).
  Prolog answer         : false.                >>> DISAGREEMENT.
    valid_move(a,c) -> FALSE <-- unsupported
```

The second block is the whole laboratory in six lines. Python says the plan is consistent with its own action model; Prolog says the action model is not consistent with the warehouse. **The verifier is right.**

Why this works, and when it would not

The two sides share **no code and no data**. The connectivity of the warehouse is written down twice — as `CONNECTED` in Python, as `connected/2` facts in Prolog — and the verifier earns its keep exactly where the two copies disagree.

Had the Prolog facts been *generated from* the Python action list, the check would have been theatre: it would agree by construction, including when both were wrong. **The independence is the mechanism, not a detail of it.**

9.6 The general architecture

Generate (fast, fallible) $\rightarrow$ Verify (narrow, trustworthy)

The verifier does not need to be able to *write* a plan — it only needs to be able to *reject* a bad one, which is a far easier job. That asymmetry is why the pattern scales: type checkers, proof assistants, model checkers and unit tests are all the same shape.

10 Answers you should be able to give

1. **Is `PickUp(Package, A)` applicable in I ?** Yes. $\text{At}(R, A)$ and $\text{At}(P, A)$ are both in I ; the negative precondition $\text{Holding}(P)$ is absent, and absent means false.
2. **Is `Drop(Package, C)` applicable in I ?** No, and it fails on *both* preconditions — the robot is not at C and is not holding anything. It is tempting to say yes because it is the action that achieves the goal and it is on the list of available actions. Neither of those is the test.
3. **What is the missing box in Task 4's diagram?** The **applicability decision**: is $S \models \text{Pre}(a)$? Everything above it is logic; everything below it is search.
4. **Where is logical reasoning used?** Four places: the applicability test $S \models \text{Pre}(a)$; the state update $S' = (S \setminus \text{Del}) \cup \text{Add}$; the goal test $S \models G$; and — outside the agent entirely — the Prolog verifier, where logic judges the agent's output rather than driving it.

5. **An error from not checking preconditions?** The planner accepts $\text{Move}(A, B)$ then $\text{PickUp}(\text{Package}, B)$ and “picks up” a package that is not there. The resulting state contains $\text{Holding}(P)$ and $\text{At}(P, A)$ — the package is in two places at once, and every later state is nonsense.
6. **Why is planning the same as the search module?** Same frontier, same visited set, same parent map, same “BFS is optimal for unit costs”. The only difference is that the successor function is $\{\text{Apply}(S, a) : a \in A, S \models \text{Pre}(a)\}$, computed by a logical test, instead of being read off a map. Two consequences: the state space is implicit and exponential (so real planners derive heuristics from the action descriptions — relaxed-plan heuristics ignore delete lists — rather than being handed an h like A^* was); and the *shape of the graph* is now something you wrote, and therefore something you can get wrong.
7. **What did the LLM contribute, and what did you verify?** It contributed the engineering: the class structure, the one-line set operations for applicability and effects, the BFS driver, the printer, and correct diagnoses once symptoms were described. Verified independently: that the plan re-executes from I ; that the planner terminates on an unsolvable problem; that the plan is shortest (against a separately written iterative-deepening oracle); and — the one that could not be done in Python at all — that the action model matches the warehouse.

11 The five words to remember

Understand $\rightarrow$ Specify $\rightarrow$ Generate $\rightarrow$ Execute $\rightarrow$ Verify

Stage	What it caught in this laboratory
Understand	that the goal is partial, so the goal test is $\subseteq$
Specify	Prompt 1 made Defect 0 a one-line fix rather than a puzzle
Generate	a correct BFS planner, quickly
Execute	Test B found Defect 1 — a hang, invisible to Test A
Verify	Prolog found Defect 2 — invisible to <i>everything</i> inside Python

The last word

An AI system can generate a candidate solution, while a separate logical system checks it. The check has to come from **outside** the thing being checked. A program that grades its own homework will pass — and so will its explanation of why it deserved to.